

Structures in C

Structures in C

A **structure** in C is a derived or user-defined data type. We use the keyword **struct** to define a custom data type that groups together the elements of different types. The difference between an array and a structure is that an **array** is a homogenous collection of similar types, whereas a structure can have elements of different types stored adjacently and identified by a name.

We are often required to work with values of different **data types** having certain relationships among them. For example, a **book** is described by its **title** (string), **author** (string), **price** (double), **number of pages** (integer), etc. Instead of using four different variables, these values can be stored in a single **struct** variable.

Declare (Create) a Structure

You can create (declare) a structure by using the **"struct"** keyword followed by the `structure_tag` (structure name) and declare all of the members of the structure inside the curly braces along with their data types.

To define a structure, you must use the **struct** statement. The struct statement defines a new data type, with more than one member.

Syntax of Structure Declaration

The format (syntax) to declare a structure is as follows –

```
struct [structure tag]{  
    member definition;  
    member definition;  
    ...  
    member definition;  
} [one or more structure variables];
```

The **structure tag** is optional and each member definition is a normal variable definition, such as "int i;" or "float f;" or any other valid variable definition.

At the end of the structure's definition, before the final semicolon, you can specify one or more structure variables but it is **optional**.

Example

In the following example we are declaring a structure for Book to store the details of a Book –

```
struct book{  
    char title[50];  
    char author[50];  
    double price;  
    int pages;  
} book1;
```

Here, we declared the structure variable **book1** at the end of the structure definition. However, you can do it separately in a different statement.

Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

Structure Variable Declaration

To access and manipulate the members of the structure, you need to declare its variable first. To declare a structure variable, write the structure name along with the "**struct**" keyword followed by the name of the structure variable. This structure variable will be used to access and manipulate the structure members.

Example

The following statement demonstrates how to declare (create) a structure variable

```
struct book book1;
```

Usually, a structure is declared before the first function is defined in the program, after the **include** statements. That way, the derived type can be used for declaring its variable inside any function.

Structure Initialization

The **initialization** of a struct variable is done by placing the value of each element inside curly brackets.

Example

The following statement demonstrates the initialization of structure

```
struct book book1 = {"Learn C", "Dennis Ritchie", 675.50, 325};
```

Accessing the Structure Members

To access the members of a structure, first, you need to declare a structure variable and then use the **dot (.) operator** along with the structure variable.

Example 1

The four elements of the struct variable book1 are accessed with the **dot (.) operator**. Hence, "book1.title" refers to the title element, "book1.author" is the author name, "book1.price" is the price, "book1.pages" is the fourth element (number of pages).

Take a look at the following example –


[Open Compiler](#)

```
#include <stdio.h>

struct book{
    char title[10];
    char author[20];
    double price;
    int pages;
};

int main(){
    struct book book1 = {"Learn C", "Dennis Ritchie", 675.50, 325};

    printf("Title:  %s \n", book1.title);
    printf("Author: %s \n", book1.author);
    printf("Price:  %lf\n", book1.price);
    printf("Pages:  %d \n", book1.pages);
    printf("Size of book struct: %d", sizeof(struct book));
    return 0;
}
```

Output

Run the code and check its output –

Title: Learn C
Author: Dennis Ritchie
Price: 675.500000
Pages: 325
Size of book struct: 48

Example 2

In the above program, we will make a small modification. Here, we will put the **type definition** and the **variable declaration** together, like this –

```
struct book{
    char title[10];
    char author[20];
    double price;
    int pages;
} book1;
```

Note that if you declare a struct variable in this way, then you cannot initialize it with curly brackets. Instead, the elements need to be assigned individually.

[Open Compiler](#)

```
#include <stdio.h>
#include <string.h>

struct book{
    char title[10];
    char author[20];
    double price;
    int pages;
} book1;

int main(){
    strcpy(book1.title, "Learn C");
    strcpy(book1.author, "Dennis Ritchie");
    book1.price = 675.50;
    book1.pages = 325;

    printf("Title: %s \n", book1.title);
```

```
printf("Author: %s \n", book1.author);
printf("Price: %lf \n", book1.price);
printf("Pages: %d \n", book1.pages);
return 0;
}
```

Output

When you execute this code, it will produce the following output –

```
Title: Learn C
Author: Dennis Ritchie
Price: 675.500000
Pages: 325
```

Copying Structures

The **assignment (=) operator** can be used to copy a structure directly. You can also use the assignment operator (=) to assign the value of the member of one structure to another.

Let's have two struct book variables, **book1** and **book2**. The variable **book1** is initialized with declaration, and we wish to assign the same values of its elements to that of **book2**.

We can assign individual elements as follows –

```
struct book book1 = {"Learn C", "Dennis Ritchie", 675.50, 325}, book2;
strcpy(book2.title, book1.title);
strcpy(book2.author, book1.author);
book2.price = book1.price;
book2.pages = book1.pages;
```

Note the use of **strcpy() function** to assign the value to a **string** variable instead of using the "=" operator".

Example

You can also assign book1 to book2 so that all the elements of book1 are respectively assigned to the elements of book2. Take a look at the following program code –



Open Compiler

```
#include <stdio.h>
#include <string.h>

struct book{
    char title[10];
    char author[20];
    double price;
    int pages;
};

int main(){
    struct book book1 = {"Learn C", "Dennis Ritchie", 675.50, 325}, book2;
    book2 = book1;

    printf("Title: %s \n", book2.title);
    printf("Author: %s \n", book2.author);
    printf("Price: %lf \n", book1.price);
    printf("Pages: %d \n", book1.pages);
    printf("Size of book struct: %d", sizeof(struct book));
    return 0;
}
```

Output

Run the code and check its output –

```
Title: Learn C
Author: Dennis Ritchie
Price: 675.500000
Pages: 325
Size of book struct: 48
```

Structures as Function Arguments

You can pass a **structure as a function argument** in the same way as you pass any other variable or pointer.

Example

Take a look at the following program code. It demonstrates how you can pass a structure as a function argument –

</>

Open Compiler

```
#include <stdio.h>
#include <string.h>

struct Books{
    char title[50];
    char author[50];
    char subject[100];
    int book_id;
};

/* function declaration */
void printBook(struct Books book);

int main(){
    struct Books Book1;          /* Declare Book1 of type Book */
    struct Books Book2;          /* Declare Book2 of type Book */

    /* book 1 specification */
    strcpy(Book1.title, "C Programming");
    strcpy(Book1.author, "Nuha Ali");
    strcpy(Book1.subject, "C Programming Tutorial");
    Book1.book_id = 6495407;

    /* book 2 specification */
    strcpy(Book2.title, "Telecom Billing");
    strcpy(Book2.author, "Zara Ali");
    strcpy(Book2.subject, "Telecom Billing Tutorial");
    Book2.book_id = 6495700;

    /* print Book1 info */
    printBook(Book1);

    /* Print Book2 info */
    printBook(Book2);
    return 0;
}
```

```
void printBook(struct Books book){

    printf("Book title : %s\n", book.title);
    printf("Book author : %s\n", book.author);
    printf("Book subject : %s\n", book.subject);
    printf("Book book_id : %d\n", book.book_id);
}
```

Output

When the above code is compiled and executed, it produces the following result –

```
Book title : C Programming
Book author : Nuha Ali
Book subject : C Programming Tutorial
Book book_id : 6495407
Book title : Telecom Billing
Book author : Zara Ali
Book subject : Telecom Billing Tutorial
Book book_id : 6495700
```

Pointers to Structures

You can define **pointers to structures** in the same way as you define **pointers** to any other variable.

Declaration of Pointer to a Structure

You can declare a pointer to a structure (or structure pointer) as follows –

```
struct Books *struct_pointer;
```

Initialization of Pointer to a Structure

You can store the address of a structure variable in the above pointer variable **struct_pointer**. To find the address of a structure variable, place the '&' operator before the structure's name as follows –

```
struct_pointer = & book1;
```


Let's store the address of a struct variable in a struct pointer variable.

```
struct book{
    char title[10];
    char author[20];
    double price;
    int pages;
};
struct book book1 = {"Learn C", "Dennis Ritchie", 675.50, 325},
struct book *strptr;
```

Accessing Members Using Pointer to a Structure

To access the members of a structure using a pointer to that structure, you must use the → **operator** as follows –

```
struct_pointer->title;
```

C defines the → symbol to be used with struct pointer as the **indirection operator** (also called **struct dereference operator**). It helps to access the elements of the struct variable to which the pointer reference to.

Example

In this example, **strptr** is a pointer to **struct book book1** variable. Hence, **strptr→title** returns the title, just like **book1.title** does.


[Open Compiler](#)

```
#include <stdio.h>
#include <string.h>

struct book{
    char title[10];
    char author[20];
    double price;
    int pages;
};

int main (){
    struct book book1 = {"Learn C", "Dennis Ritchie", 675.50, 325};
```

```

struct book *strptr;
strptr = &book1;

printf("Title: %s \n", strptr -> title);
printf("Author: %s \n", strptr -> author);
printf("Price: %lf \n", strptr -> price);
printf("Pages: %d \n", strptr -> pages);
return 0;
}

```

Output

When you run this code, it will produce the following output –

```

Title: Learn C
Author: Dennis Ritchie
Price: 675.500000
Pages: 325

```

Note: The dot (.) operator is used to access the struct elements via the **struct** variable. To access the elements via its pointer, we must use the **indirection** (->) operator

A struct variable is like a normal variable of primary type, in the sense that you can have an array of struct, you can pass the struct variable to a function, as well as return a struct from a function.

You may have noted that you need to prefix "struct type" to the name of the variable or pointer at the time of declaration. This can be avoided by creating a shorthand notation with the help of **typedef** keyword, which we will explain in a subsequent chapter.

Structures are used in different applications such as databases, file management applications, and for handling complex data structures such as tree and linked lists.

Bit Fields

Bit Fields allow the packing of data in a structure. This is especially useful when memory or data storage is at a premium. Typical examples include –

- Packing several objects into a machine word, for example, 1-bit flags can be compacted.

- Reading external file formats – non-standard file formats could be read in, for example, 9-bit integers.

Declaration

C allows us to do this in a structure definition by putting :bit length after the variable. For example –

```
struct packed_struct{
    unsigned int f1:1;
    unsigned int f2:1;
    unsigned int f3:1;
    unsigned int f4:1;
    unsigned int type:4;
    unsigned int my_int:9;
} pack;
```

Here, the packed_struct contains 6 members: Four 1 bit flags f1..f3, a 4-bit type and a 9-bit my_int.

C automatically packs the above bit fields as compactly as possible, provided that the maximum length of the field is less than or equal to the integer word length of the computer. If this is not the case, then some compilers may allow memory overlap for the fields while others would store the next field in the next word.

Structures and Functions in C

In C programming, **struct** is a derived data type. Just as we can pass arguments of primary data types, a variable of struct data type can also be passed to a function. You can also pass **structures** using **call by value** and **call by reference** methods. A function in C may also return a struct data type.

Read this chapter to understand the following concepts –

- How to pass elements of struct type
- How to pass a struct variable
- How to return struct from a function
- How to return a struct pointer

Let's start with the first one and learn how to pass elements of struct type.

How to Pass Struct Elements

A derived type is a combination of one or more elements of any of the primary types as well as another derived type. It is possible to pass elements to a **function**, either by value or by reference.

Example

In the following example, we have a derived type called "rectangle" with two elements. We have a struct variable "r" with the elements "r.len" and "l.brd" and they are passed to a function. The area() function then computes the area of the rectangle.


[Open Compiler](#)

```
#include <stdio.h>

struct rectangle{
    float len, brd;
};

int area(float, float);

int main(){
```

```

    struct rectangle r;
    r.len = 10.50; r.brd = 20.5;
    area(r.len, r.brd);

    return 0;
}

int area(float a, float b){

    double area = (double)(a*b);
    printf("Length: %f \nBreadth: %f \nArea: %lf\n", a, b, area);

    return 0;
}

```

Output

When you run this code, it will produce the following output –

```

Length: 10.500000
Breadth: 20.500000
Area: 215.250000

```

How to Pass a Struct Variable

Let us modify the above example to pass the struct variable itself (instead of its elements) to the area() function. The rectangle struct type also has an additional element called "area".

Example

Inside the function, the elements of the struct variable are accessed though the dot operator (.) and the area is calculated.


[Open Compiler](#)

```

#include <stdio.h>

struct rectangle{
    float len, brd;

```

```
double area;
};

int area(struct rectangle);

int main(){

    struct rectangle r;
    r.len = 10.50; r.brd = 20.5;
    area(r);

    return 0;
}

int area(struct rectangle r){

    r.area = (double)(r.len*r.brd);
    printf("Length: %f \nBreadth: %f \nArea: %lf\n", r.len, r.brd, r.area);

    return 0;
}
```

Output

Run the code and check its output –

```
Length: 10.500000
Breadth: 20.500000
Area: 215.250000
```

Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

How to Return Struct from a Function

We know that a function in C can return a value of any type. In this example, the area() function is defined to return a struct variable.

Example

Inside the main() fuction, the inputs for the length and the breadth are passed to the area() function. Inside the area() function, the area is computed and a struct variable is

populated and returned to the main() function, where its elements are displayed.

</>

Open Compiler

```
#include <stdio.h>

struct rectangle {
    float len, brd;
    double area;
};

struct rectangle area(float x, float y);

int main(){

    struct rectangle r;
    float x, y;

    x = 10.5; y = 20.5;
    r = area(x, y);

    printf("Length: %f \n Breadth: %f \n Area: %lf\n", r.len, r.brd, r.area);

    return 0;
}

struct rectangle area(float x, float y){

    double area = (double)(x*y);
    struct rectangle r = {x, y, area};

    return r;
}
```

Output

When you run this code, it will produce the following output –

```
Length: 10.500000
Breadth: 20.500000
```

Area: 215.250000

How to Pass a Struct by Reference

In C language, a function may be defined to have its arguments passed by value or reference. A reference is the **pointer** to an existing variable.

Example

In this example, a struct variable of "rectangle" type is declared in main() and its address is passed to a user-defined function called area().

When the area() function is called, it can use the elements of the variable with the indirection operator ($\rightarrow$). It computes the result and assigns it to the area element "r $\rightarrow$ area".


[Open Compiler](#)

```
#include <stdio.h>

struct rectangle{
    float len, brd;
    double area;
};

int area(struct rectangle *);

int main(){

    struct rectangle r;
    r.len = 10.50; r.brd = 20.5;
    area(&r);

    return 0;
}

int area(struct rectangle *r){

    r -> area = (double)(r -> len * r -> brd);
    printf("Length: %f \nBreadth: %f \nArea: %lf\n", r -> len, r -> brd, r ->
area);
}
```



```
return 0;
}
```

Output

Run the code and check its output –

```
Length: 10.500000
Breadth: 20.500000
Area: 215.250000
```

How to Return a Struct Pointer

Let us rewrite the above code to define the area() function and return a pointer to a struct of rectangle data type.

Example

The area() function has two call-by-value arguments. The main() function reads the length and breadth from the user and passes them to the area() function, which populates a struct variable and passes its reference back to the main() function.


[Open Compiler](#)

```
#include <stdio.h>

struct rectangle {
    float len, brd;
    double area;
};

struct rectangle * area(float x, float y);

int main (){

    struct rectangle *r;
    float x, y;

    x = 10.5; y = 20.5;
    r = area(x, y);
```

```
printf("Length: %f \n Breadth: %f \n Area: %lf\n", r->len, r->brd, r->area);

return 0;
}

struct rectangle * area(float x, float y){

    double area = (double)(x*y);
    static struct rectangle r;
    r.len = x; r.brd = y; r.area = area;

    return &r;
}
```

Output

When you run this code, it will produce the following output –

```
Length: 10.500000
Breadth: 20.500000
Area: 215.250000
```

Structures and Functions in C

In C programming, **struct** is a derived data type. Just as we can pass arguments of primary data types, a variable of struct data type can also be passed to a function. You can also pass **structures** using **call by value** and **call by reference** methods. A function in C may also return a struct data type.

Read this chapter to understand the following concepts –

- How to pass elements of struct type
- How to pass a struct variable
- How to return struct from a function
- How to return a struct pointer

Let's start with the first one and learn how to pass elements of struct type.

How to Pass Struct Elements

A derived type is a combination of one or more elements of any of the primary types as well as another derived type. It is possible to pass elements to a **function**, either by value or by reference.

Example

In the following example, we have a derived type called "rectangle" with two elements. We have a struct variable "r" with the elements "r.len" and "l.brd" and they are passed to a function. The area() function then computes the area of the rectangle.


[Open Compiler](#)

```
#include <stdio.h>

struct rectangle{
    float len, brd;
};

int area(float, float);

int main(){
```

```

    struct rectangle r;
    r.len = 10.50; r.brd = 20.5;
    area(r.len, r.brd);

    return 0;
}

int area(float a, float b){

    double area = (double)(a*b);
    printf("Length: %f \nBreadth: %f \nArea: %lf\n", a, b, area);

    return 0;
}

```

Output

When you run this code, it will produce the following output –

```

Length: 10.500000
Breadth: 20.500000
Area: 215.250000

```

How to Pass a Struct Variable

Let us modify the above example to pass the struct variable itself (instead of its elements) to the area() function. The rectangle struct type also has an additional element called "area".

Example

Inside the function, the elements of the struct variable are accessed though the dot operator (.) and the area is calculated.


[Open Compiler](#)

```

#include <stdio.h>

struct rectangle{
    float len, brd;

```

```
double area;
};

int area(struct rectangle);

int main(){

    struct rectangle r;
    r.len = 10.50; r.brd = 20.5;
    area(r);

    return 0;
}

int area(struct rectangle r){

    r.area = (double)(r.len*r.brd);
    printf("Length: %f \nBreadth: %f \nArea: %lf\n", r.len, r.brd, r.area);

    return 0;
}
```

Output

Run the code and check its output –

```
Length: 10.500000
Breadth: 20.500000
Area: 215.250000
```

Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

How to Return Struct from a Function

We know that a function in C can return a value of any type. In this example, the area() function is defined to return a struct variable.

Example

Inside the main() fuction, the inputs for the length and the breadth are passed to the area() function. Inside the area() function, the area is computed and a struct variable is

populated and returned to the main() function, where its elements are displayed.

</>

Open Compiler

```
#include <stdio.h>

struct rectangle {
    float len, brd;
    double area;
};

struct rectangle area(float x, float y);

int main(){

    struct rectangle r;
    float x, y;

    x = 10.5; y = 20.5;
    r = area(x, y);

    printf("Length: %f \n Breadth: %f \n Area: %lf\n", r.len, r.brd, r.area);

    return 0;
}

struct rectangle area(float x, float y){

    double area = (double)(x*y);
    struct rectangle r = {x, y, area};

    return r;
}
```

Output

When you run this code, it will produce the following output –

```
Length: 10.500000
Breadth: 20.500000
```

Area: 215.250000

How to Pass a Struct by Reference

In C language, a function may be defined to have its arguments passed by value or reference. A reference is the **pointer** to an existing variable.

Example

In this example, a struct variable of "rectangle" type is declared in main() and its address is passed to a user-defined function called area().

When the area() function is called, it can use the elements of the variable with the indirection operator ($\rightarrow$). It computes the result and assigns it to the area element "r $\rightarrow$ area".


[Open Compiler](#)

```
#include <stdio.h>

struct rectangle{
    float len, brd;
    double area;
};

int area(struct rectangle *);

int main(){

    struct rectangle r;
    r.len = 10.50; r.brd = 20.5;
    area(&r);

    return 0;
}

int area(struct rectangle *r){

    r -> area = (double)(r -> len * r -> brd);
    printf("Length: %f \nBreadth: %f \nArea: %lf\n", r -> len, r -> brd, r ->
area);
}
```

```
return 0;
}
```

Output

Run the code and check its output –

```
Length: 10.500000
Breadth: 20.500000
Area: 215.250000
```

How to Return a Struct Pointer

Let us rewrite the above code to define the area() function and return a pointer to a struct of rectangle data type.

Example

The area() function has two call-by-value arguments. The main() function reads the length and breadth from the user and passes them to the area() function, which populates a struct variable and passes its reference back to the main() function.


[Open Compiler](#)

```
#include <stdio.h>

struct rectangle {
    float len, brd;
    double area;
};

struct rectangle * area(float x, float y);

int main (){

    struct rectangle *r;
    float x, y;

    x = 10.5; y = 20.5;
    r = area(x, y);
```



```
printf("Length: %f \n Breadth: %f \n Area: %lf\n", r->len, r->brd, r->area);

return 0;
}

struct rectangle * area(float x, float y){

    double area = (double)(x*y);
    static struct rectangle r;
    r.len = x; r.brd = y; r.area = area;

    return &r;
}
```

Output

When you run this code, it will produce the following output –

```
Length: 10.500000
Breadth: 20.500000
Area: 215.250000
```